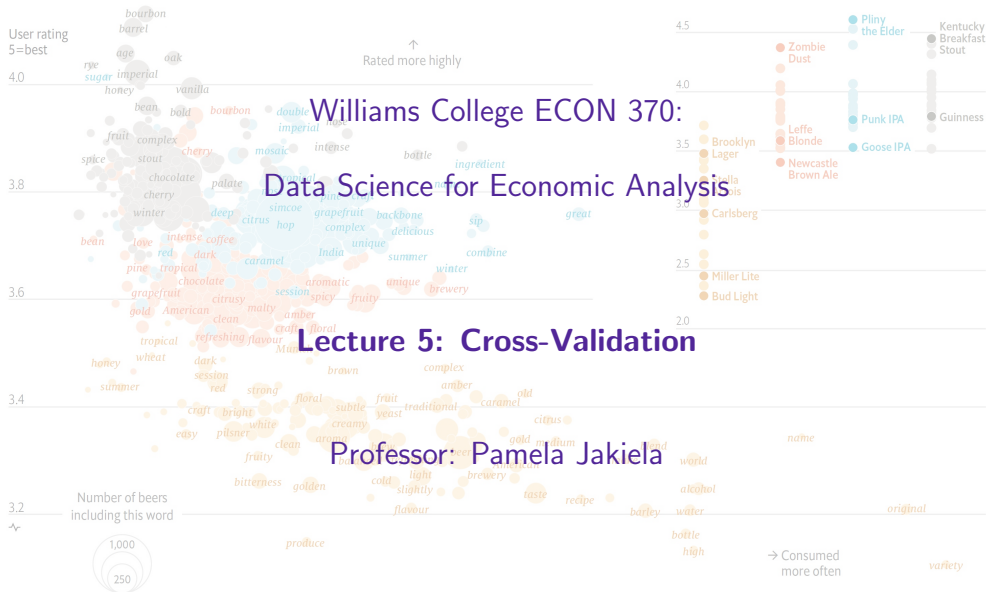




Outline

- What is over-fitting, and why is it a problem?
- Training data vs. test data and test mean squared error
- Using cross-validation to choose an OLS specification

Thought Experiment: 100 True Null Hypotheses

Let Y and $X_1, \dots, X_{100}$ be column vectors containing $N = 20$ draws from iid standard normals.

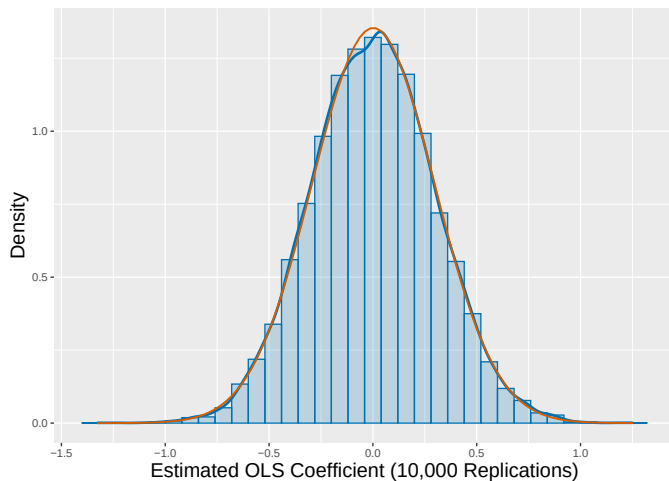
Suppose you run 100 regressions, regressing Y on each X_i variable (individually).

What is the expected value of $\hat{\beta}_{OLS}$?

How many of the estimated OLS coefficients should we expect to be statistically significant?

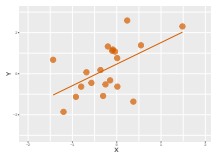
What if we ran 10,000 regressions? How many significant X_i variables should we expect?

$\hat{\beta}_{OLS} \sim N(\beta_0, \tilde{\sigma}_{OLS})$: OLS Coefficients Are Normally Distributed About β_0

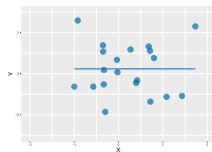


Strong Associations Occur by Chance (with 10,000 Replications)

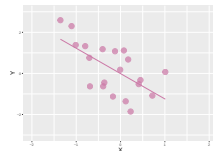
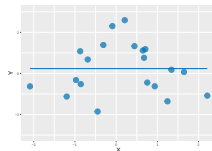
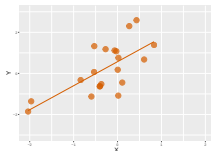
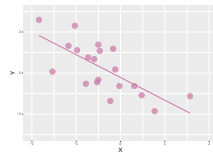
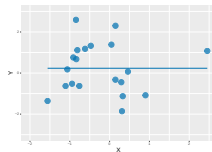
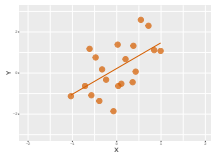
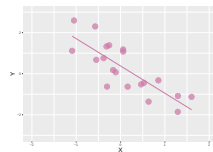
Top 3



Middle 3



Bottom 3



Over-Fitting

Why does it matter that OLS (or any other statistical procedure) is fitting the noise?

- Inference: if you test a large number of hypotheses, some will be rejected (wrongly)
 - ▶ You may falsely conclude that two variables X and Y are correlated, or that policy X impacts outcome Y , or that outcome Y is different for two groups (e.g. men vs. women)
- Prediction: a less flexible model that ties our hands and prevents us from fitting the noise often provides better out-of-sample predictions than a more flexible (higher R^2) model
 - ▶ Using an overly complex or flexible model that ends up explaining both the underlying data-generating process and random error terms is often referred to as **over-fitting**

Over-fitting becomes a serious concern with many data science techniques that make use of very large data sets (including not just large N s but many variables) and very flexible models

- (Aside about prediction vs. inference/causality and statistics/CS vs. economics here)

Another Thought Experiment: 100 True Null Hypotheses

Let Y and $X_1, \dots, X_{100}$ be column vectors containing $N = 20$ draws from iid standard normals.

Split the data in half into Data Set 1 ($N = 10$) and Data Set 2 ($N = 10$).

Run 100 regressions, regressing Y on each X_i variable (individually) using only Data Set 1.

How many of the estimated OLS coefficients should we expect to be statistically significant?

How many of the X_i variables that are statistically significant predictors of Y in Data Set 1 will also be statistically significant predictors of Y in Data Set 2?

The Validation Set Approach to Specification Selection

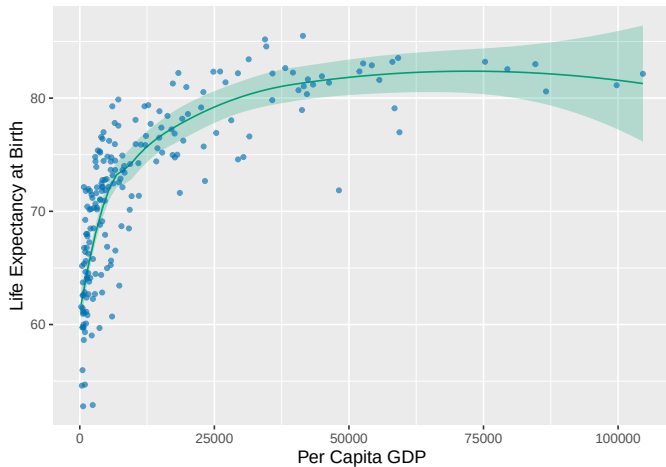
OLS minimizes the residual sum of squares (RSS) or, equivalently, the mean squared error (MSE), but if we care about prediction then we want to minimize the MSE in **new data**

- OLS will never tell you that you have included too many (non-collinear) variables

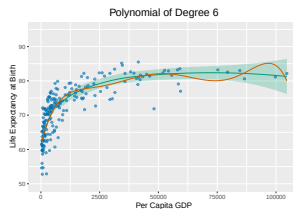
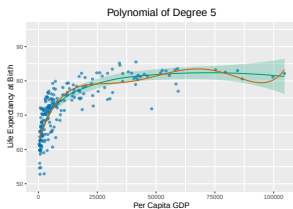
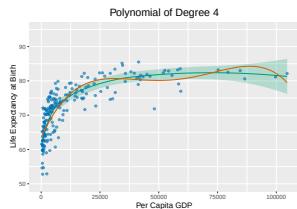
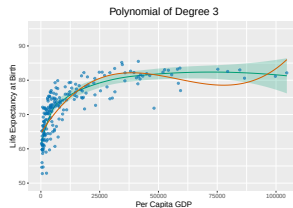
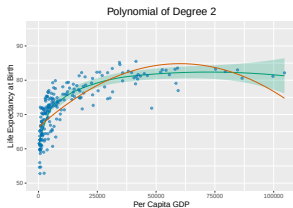
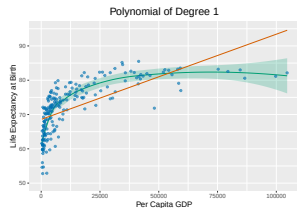
We don't always care about prediction, but when we do we need a strategy to avoid over-fitting

- One approach is to randomly partition the data into a **training** sample and a **test** sample
 - ▶ The training sample is used to choose or fit the model
 - ▶ The test sample is used to run the model, calculate the RSS or MSE
- The validation set approach builds on the intuition from the thought experiment: random noise in the training, test data sets is uncorrelated, so if we fit the model on the training data and then calculate the MSE on the test data, there is no (ok, much less) over-fitting

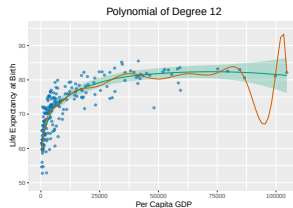
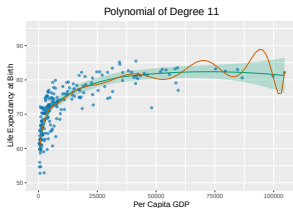
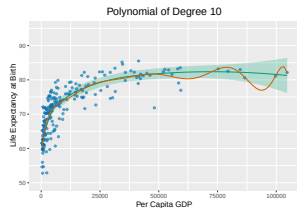
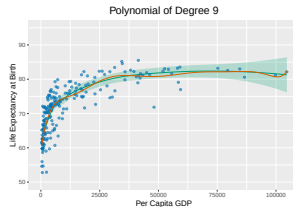
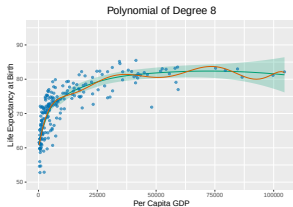
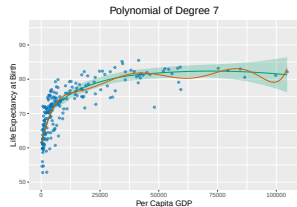
Example: The Preston Curve



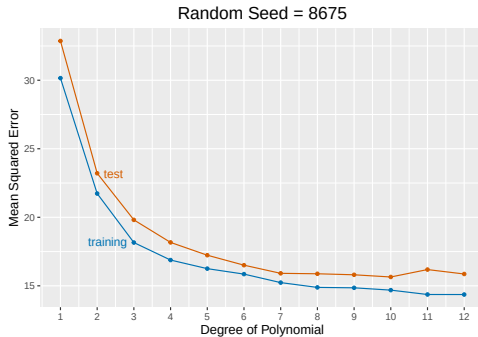
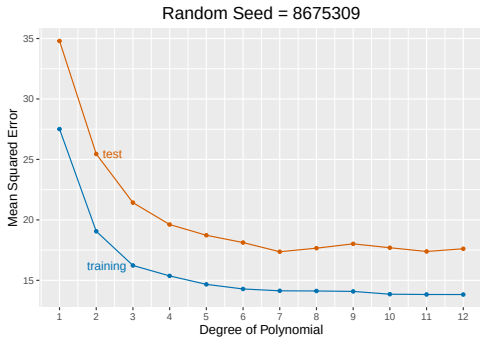
Over-Fitting the Preston Curve



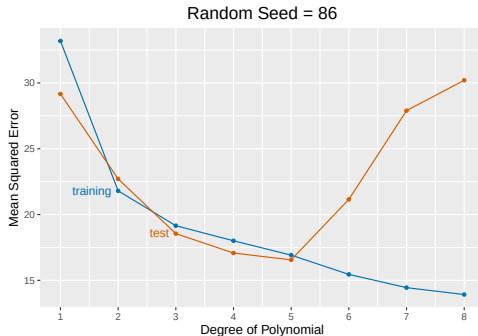
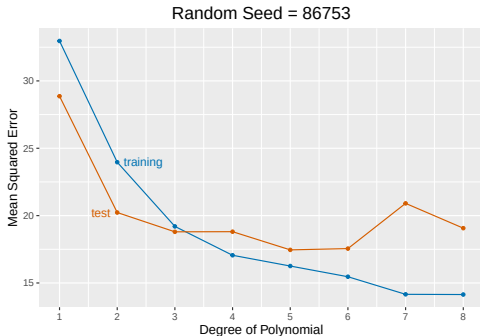
Over-Fitting the Preston Curve



The Validation Set Approach in Practice: Preston Curve Data

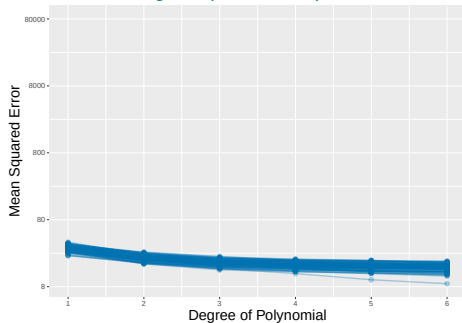


The Validation Set Approach in Practice: Preston Curve Data

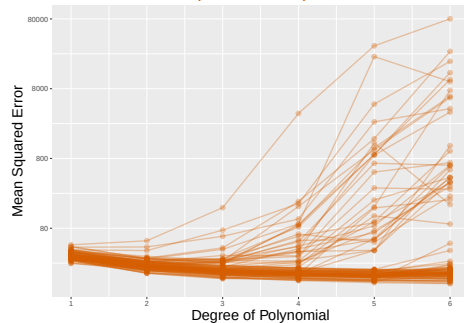


The Validation Set Approach in Practice: 100 Replications

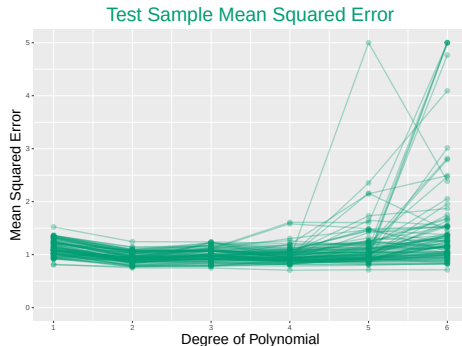
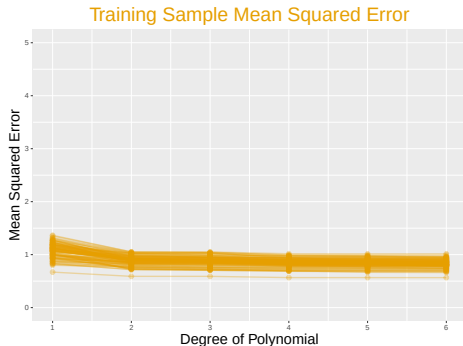
Training Sample Mean Squared Error



Test Sample Mean Squared Error



Validation Set Approach Correct 43% of Time in Simulated Data*



data-generating process: $Y = 0.8X + 0.4X^2 + \varepsilon$ where $X \sim N(0, 1)$, $\varepsilon \sim N(0, 1)$, $N = 240$

*This is just one example, but the validation set approach is often noisy

Cross-Validation

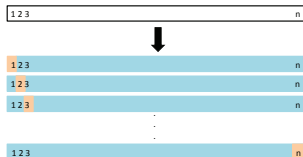
In **cross-validation**, we run the validation set approach multiple times and average the results

- Each observation only appears in the test data set once
- In each cut of the data, all the observations that are not test data are training data

Two types of cross-validation, defined by the way we construct the test data sets:

- Leave-One-Out Cross-Validation (LOOCV): each observation is a singleton test data set
- k -Fold Cross-Validation: partition the data into k **folds**, each a test data set

Leave-One-Out Cross-Validation

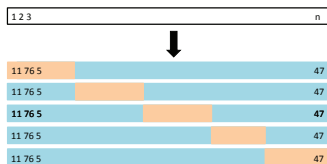


source: James et al. (2021)

Leave-One-Out Cross-Validation (LOOCV): each observation is a singleton test data set

- Run the validation set approach N times using all other observations as training data
- Calculate the squared prediction error for each observation, average to calculate MSE
- LOOCV may be computationally costly (when using models more complicated than OLS)

k -Fold Cross-Validation

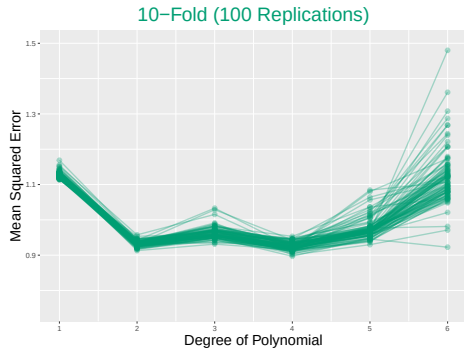
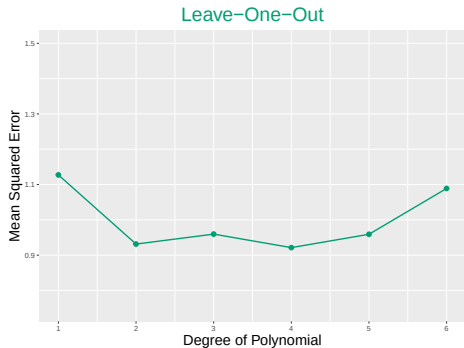


source: James et al. (2021)

k -Fold Cross-Validation:

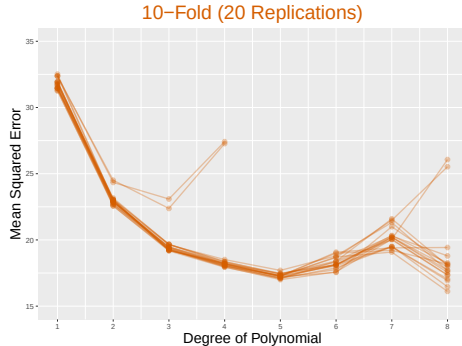
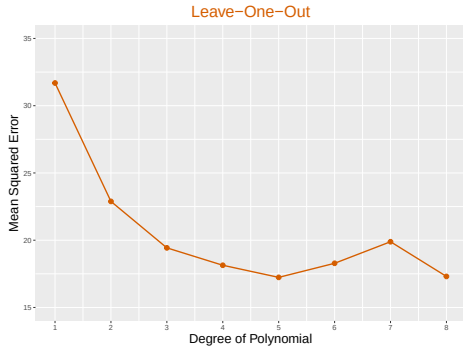
- Randomly partition the sample into k distinct test data sets (aka folds)
- Run the validation set approach k times using the observations in fold k as the test data and the remainder of the data (i.e. everything but fold k) as the associated training data
- Calculate the mean squared prediction error in each fold, average across folds

Comparing Test MSE: Leave-One-Out CV vs. k -Fold CV



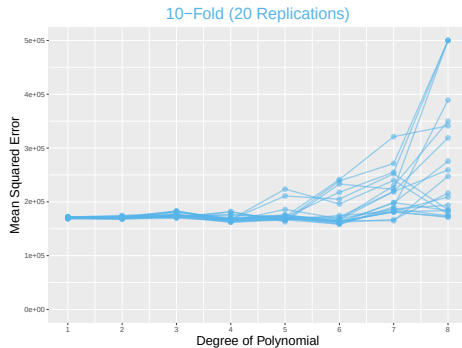
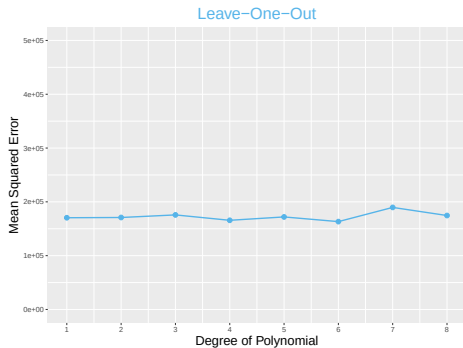
data-generating process: $Y = 0.8X + 0.4X^2 + \varepsilon$ where $X \sim N(0, 1)$, $\varepsilon \sim N(0, 1)$, $N = 240$

Comparing Test MSE: Leave-One-Out CV vs. k -Fold CV



data source: World Bank WDI data on GDP per capita and life expectancy at birth

Comparing Test MSE: Leave-One-Out CV vs. k -Fold CV



data source: James et al. (2021) hitters data on runs and salary, a case where polynomial terms do not improve fit

Summary: Cross-Validation to Choose an OLS Specification

Cross-validation allows us to limit over-fitting when choosing a specification

- Randomize observations into k distinct groups (folds), make sure to set the seed
 - ▶ Skip this step in LOOCV
- For each of the k (in k -fold) or N (in LOOCV) partitions of the data into training vs. test:
 - ▶ Estimate each candidate specification j in the training data
 - ▶ Use the estimated coefficients to predict outcomes in test data in fold k
 - ▶ Calculate squared residuals, MSE in fold k for candidate specification j
- Calculate test MSE for specification j by averaging across folds (or test data sets)
- If graph of test MSEs has a flat bottom, tend toward most parsimonious specification

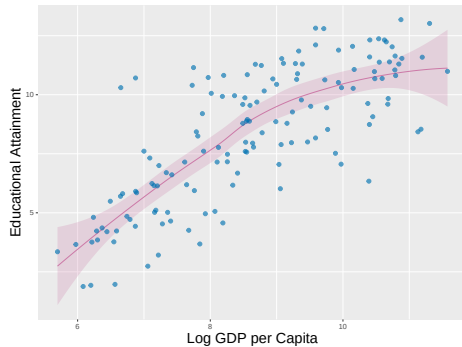
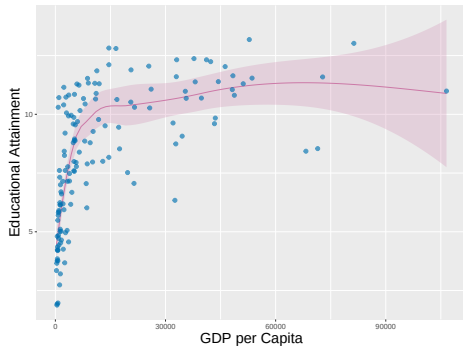
Lab #5

Objective: in this lab, you will implement k -fold cross-validation (with $k = 10$) using the data on GDP per capita and educational attainment from lab #1

You will find the polynomial function of log GDP that best predicts educational attainment

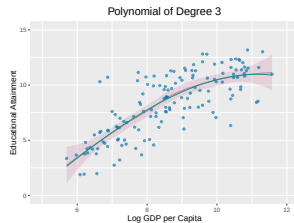
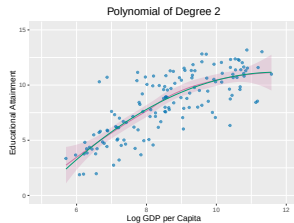
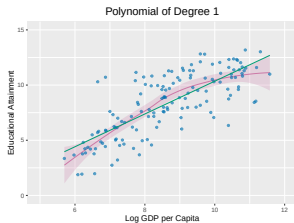
You will not use any packages or pre-built functions that implement cross-validation

Lab #5



data source: merge data from lab #1 ($N = 143$), WDI data on GDP and Barro-Lee data on education

Lab #5



data source: merge data from lab #1 ($N = 143$), WDI data on GDP and Barro-Lee data on education

Lab #5 in Four Steps

0. Load the data and familiarize yourself with it (you will regress `mean_edu` on `log_gdp`)
1. Fit the model with 2 polynomial terms; calculate test and train MSE
2. Loop through different numbers of polynomial terms (1 through 8)
3. Implement 10-fold cross-validation
4. Graph your results

Splitting the Sample into Training and Test Data

```
datasize = 8  
num_folds = 2
```

folds: $\begin{bmatrix} 1 \\ 2 \\ 1 \\ 2 \\ 1 \\ 2 \\ 1 \\ 2 \end{bmatrix} \Rightarrow \begin{bmatrix} 1 \\ 1 \\ 2 \\ 1 \\ 2 \\ 2 \\ 2 \\ 2 \end{bmatrix}$

Training data: folds == 1

- Fit OLS regression
- Calculate training MSE using residuals
- Calculate “check” MSE by predicting Y

Test data: folds != 1

- Predict Y using coefficients from training data
- Calculate test MSE (“by hand”)

Polynomial Regression

R:

```
ols <- lm(mean_edu ~ poly(log_gdp, 2, raw = TRUE), data = train_data)
...
test_xmat <- tibble(rep(1, length(test_data$mean_edu)), poly(test_data$log_gdp, 2, raw = TRUE))
```

Python:

```
from sklearn.preprocessing import PolynomialFeatures
...
poly = PolynomialFeatures(degree=2, include_bias=False)
X_train_poly = poly.fit_transform(train_data[['log_gdp']])
...
X_test_poly = poly.transform(test_data[['log_gdp']])
test_yhat = model.predict(sm.add_constant(X_test_poly))
```

Polynomial Regression

```
datasize = 143
```

```
max_order = 8
```

```
poly_id =  $\begin{bmatrix} 1 \\ 2 \\ 3 \\ 4 \\ 5 \\ 6 \\ 7 \\ 8 \end{bmatrix}$ 
```

Polynomial Regression

`datasize = 143`

`max_order = 8`

`poly_id =` $\begin{bmatrix} 1 \\ 2 \\ 3 \\ 4 \\ 5 \\ 6 \\ 7 \\ 8 \end{bmatrix}$ `test_mse =` $\begin{bmatrix} i \end{bmatrix}$

Cross-Validation

```
datasize = 143
```

```
max_order = 2
```

```
num_folds = 3
```

```
fold_id =  $\begin{bmatrix} 1 \\ 1 \\ 2 \\ 2 \\ 3 \\ 3 \end{bmatrix}$ 
```

```
poly_id =  $\begin{bmatrix} 1 \\ 2 \\ 1 \\ 2 \\ 1 \\ 2 \end{bmatrix}$ 
```

```
test_mse =  $\begin{bmatrix} j \\ \\ \\ \\ \\ \end{bmatrix}$ 
```

$j = \text{max_order} * (n - 1) + i$

where n is iteration of fold loop and

i is iteration of polynomial loop

Cross-Validation: Results

